

多模态生成技术优化实践第二期： 并行和Cache篇

作者：王正平，倪家楠，具昌浩

时间：2026/05/14

HunyuanVideo

HunyuanVideo是一款DiT架构的视频生成模型，DiT网络参数量13B



cann recipes infer



<https://gitcode.com/cann>

CANN

优化总览

HunyuanVideo的昇腾NPU适配项目已在cann-recipes-infer仓开源，通过叠加序列并行和FeatureCache，可实现14.40倍端到端加速！



cann recipes infer

视频规格：1280*720P 129帧

并行度	Cache方法	DiT耗时(s)	e2e耗时(s)	DiT加速	e2e加速
1	x	3568.36	3612.3	1.00	1.00
	FBCache	1693.89	1778.28	2.11	2.03
	TeaCache	1737.92	1830.75	2.05	1.97
	TaylorSeer	1405.68	1500.17	2.54	2.41
Ulysses 8	x	465.43	487.68	7.67	7.41
	FBCache	235.8	256.5	15.13	14.08
	TeaCache	290.79	305.74	12.27	11.81
	TaylorSeer	232.84	250.83	15.33	14.40

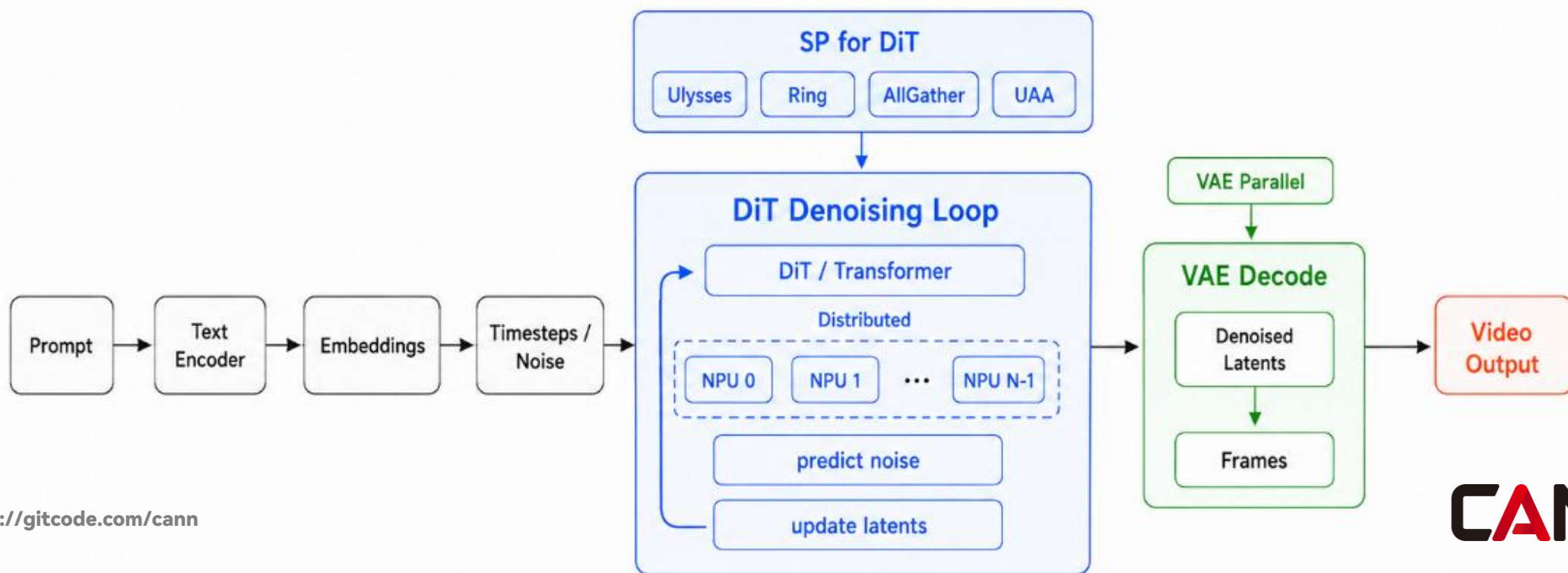
目录

Part 1 Sequence Parallel

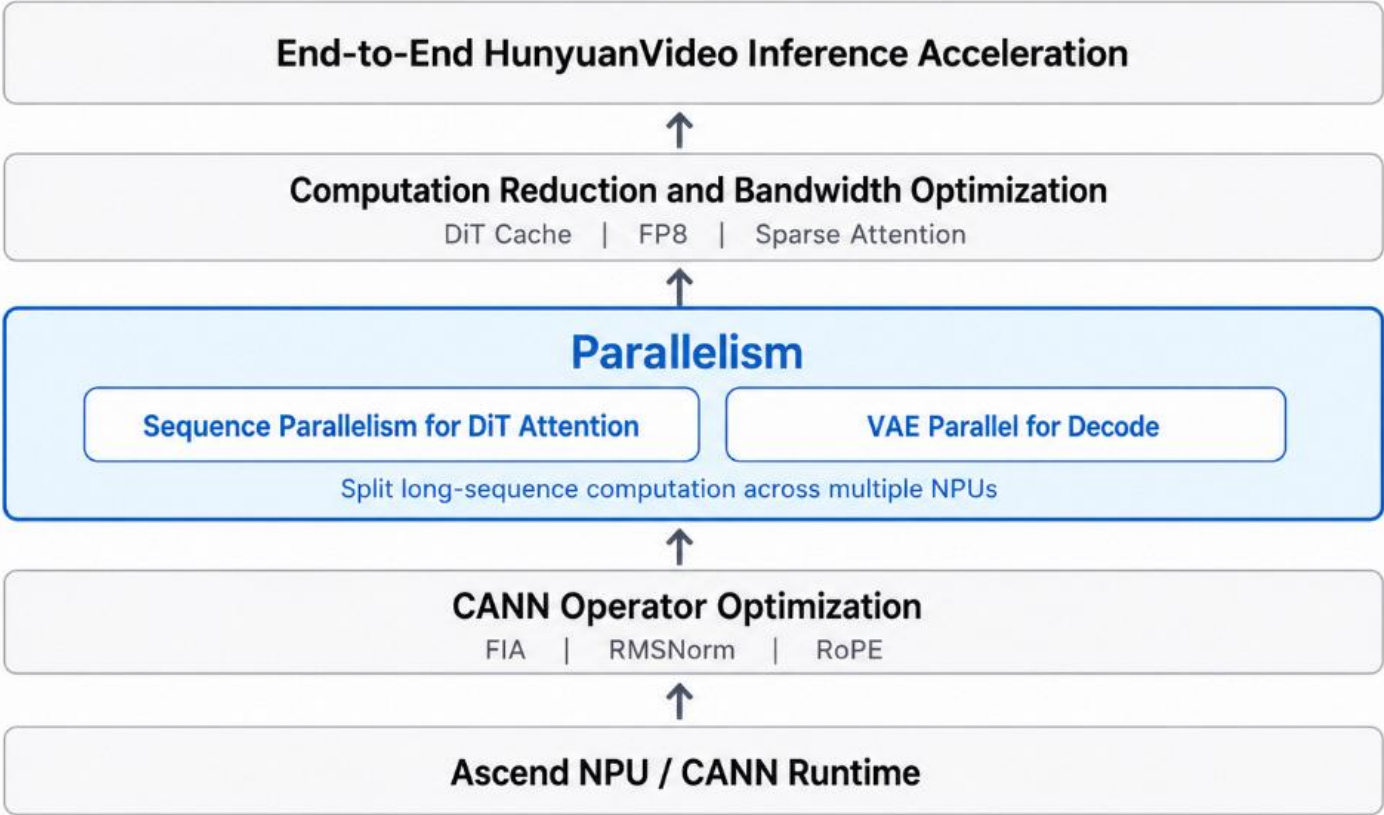
Part 2 Feature Cache

并行切分——以HunyuanVideo的NPU推理优化为例

- 面向多模态生成模型算法与推理部署开发者
- 以 HunyuanVideo 为例
- 聚焦多模生成推理中的多卡并行切分
- 介绍 recipes infer 中已有并行优化样例与配置方式



并行切分不是孤立的优化点



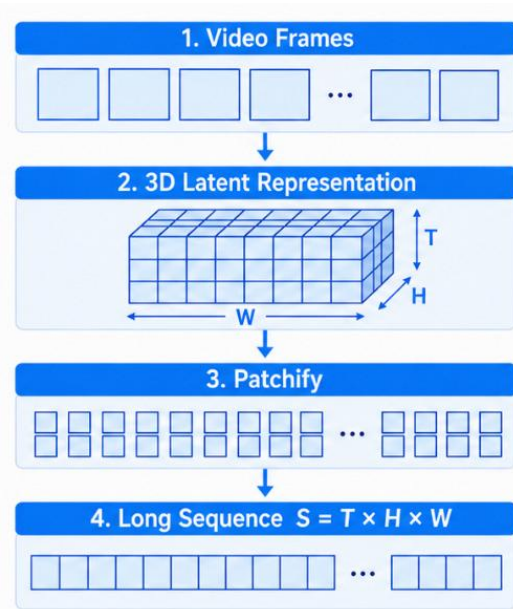
多模态生成推理优化可以分为多个层次：

优化方向	主要作用
CANN 融合算子	提升基础算子效率
多模态并行切分	分摊长序列显存与计算压力
特征 Cache	减少扩散过程中的重复计算
FP8 / 稀疏 Attention	降低带宽和计算量
VAE 并行	优化视频解码阶段

并行解决的是：已经确定要算的**长序列计算**，如何拆到**多张卡**上高效执行

多模生成推理的并行需求

多模生成网络和常规LLM推理的压力结构不同。以视频生成为例，其多模网络会同时展开时间维和空间维，最终形成非常长的 latent 序列；并行在这里会作为一种提升视频生成吞吐、降低端到端时延、改善显存与计算压力的手段。



119K Sequence

5s 720p 3D full attention scale

95% DiT Time

DiT dominates end-to-end latency

81% Attention Time

Attention is the key bottleneck



并行主要作用在 DiT 长序列计算, DiT 需要重复执行多步, 每一步都包含**长序列 Attention**、MLP、Norm 和 Projection 等计算。

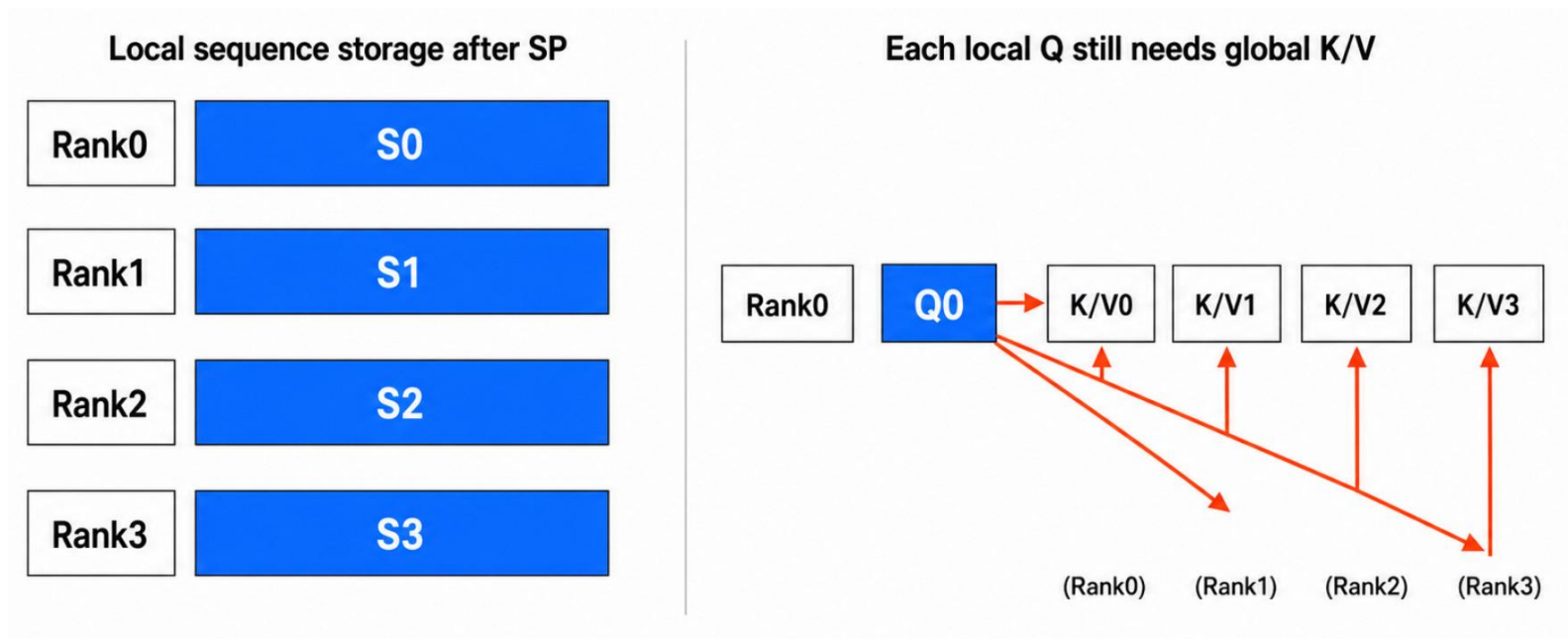
并行到底在拆什么？

大模型并行本质上是在合适的维度上拆计算。可以拆 batch、拆参数、拆序列，也可以拆扩散模型中的条件分支和无条件分支。对视频生成网络来说，最核心的压力来自超长时空序列，因此**序列并行是主线**。

并行方式	拆分维度	在多模态生成中的作用
数据并行 DP	Batch	更适合多请求吞吐，单请求收益有限
张量并行 TP	参数 / 矩阵	可用于大模型权重和矩阵计算切分
序列并行 SP	Sequence	解决长视频序列压力，是本节重点
CFG 并行	条件 / 无条件分支	降低双分支串行前向时延
VAE 并行	Latent 空间维度	加速视频解码阶段

序列能切开，Attention 语义不能断

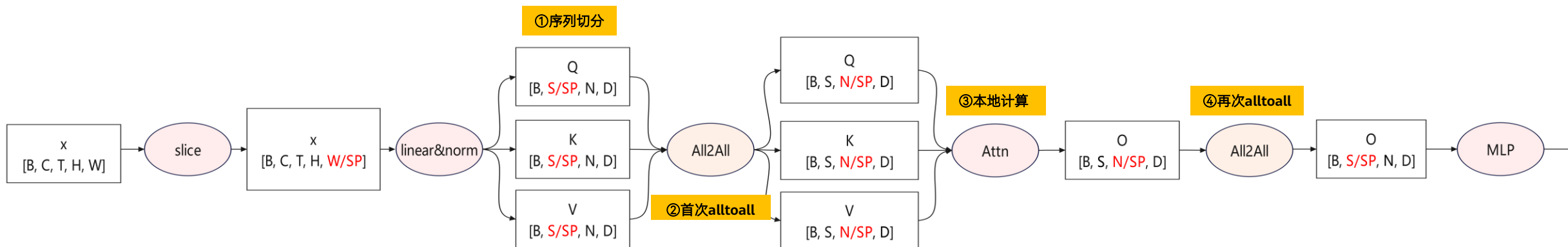
我们可以很自然地把长序列沿 S 维切开，每张卡只保留 S/SP 的局部序列。但是**存储上可以切开，语义上不能切断**。所以会引入一个问题，**远端 K/V 需要怎么去拿？**



Ulysses、Ring Attention等切分方法，
本质上都是在回答这同一个问题

序列并行：Ulysses

Attention 外切序列，Attention 内切 Head



- ① 序列切分：外部按序列切分，每卡持有 $[B, S/SP, N, D]$ 。
- ② 首次 All-to-All：进 Attention 前换维，布局变为 $[B, S, N/SP, D]$ ，每卡拿完整序列、部分 Head。
- ③ 本地计算：Head 间天然独立，直接本地计算 Attention。
- ④ 再次 All-to-All：计算后恢复序列切分 $[B, S/SP, N, D]$ ，继续后续 Block。

在单机多卡场景下，Ulysses 工程路径清晰，通常优先考虑。但标准 Ulysses 也有约束：

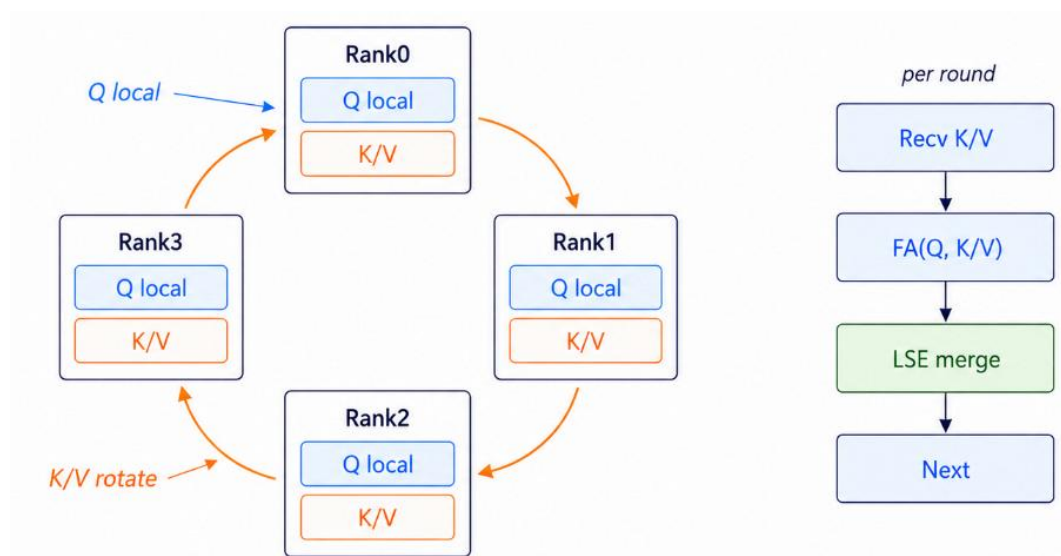
- 1. Head 数需被 SP 整除 (如 HyVideo 为 24，需据此选择并行度)；
- 2. 视频尺寸对应的 latent 切分维度最好也能被 SP 整除。

[2309.14509] DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models

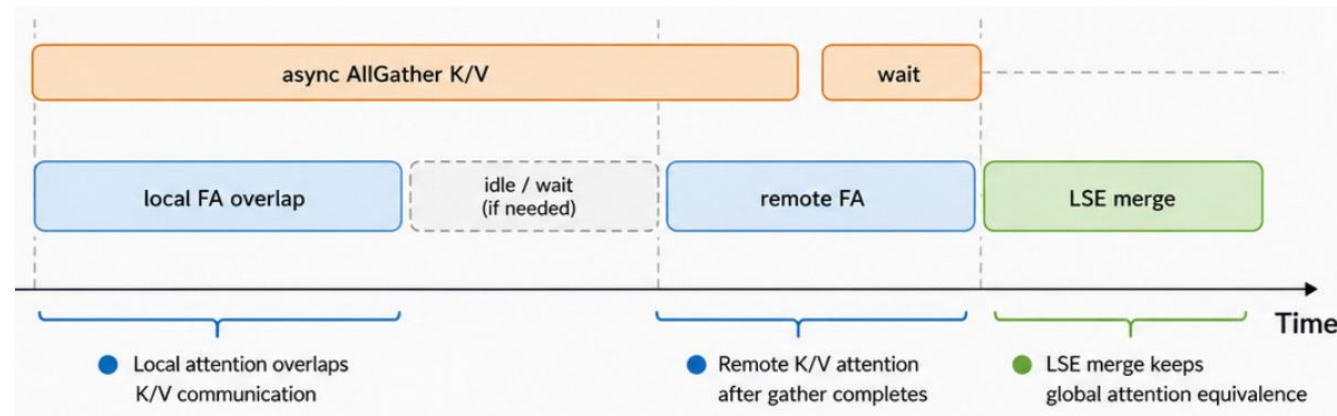
<https://gitcode.com/cann>

CANN

序列并行：Ring Attention



传统Ring Attention 的优势是**显存低、单轮 FA 规模小**；代价是**多轮 p2p 通信的效率往往没有集合通信高**。



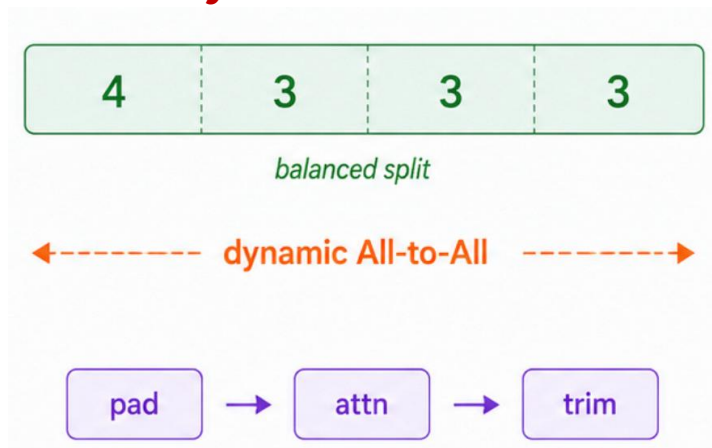
Overlap AllGatherSP版：异步发起 AllGather，在通信期间先计算本地 K/V 对应的 **attention**；通信完成后，再计算远端 K/V 对应的 **attention**，最后用 LSE 合并结果。

[2310.01889] Ring Attention with Blockwise Transformers for Near-Infinite Context

<https://gitcode.com/cann>

序列并行：UAA & USP

让 Ulysses 支持不规整输入



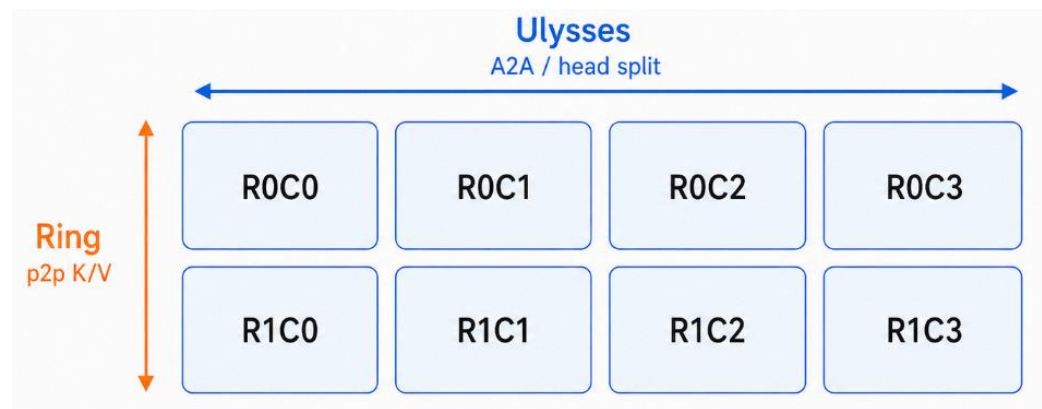
UAA，也就是 Ulysses Anything Attention，主要解决工程泛化问题。对于序列长度不能均匀切分的输入，UAA **采用更均衡的切分方式**，并在通信前收集各 rank 的真实长度，再做动态 All-to-All。对于 **head 数不能整除并行度的情况**，它会在 **head 维度 padding**，attention 后再去掉 padding。

UAA: Ulysses Anything Attention ([Context Parallelism - Cache-DiT](#))

USP: A Unified Sequence Parallelism Approach for Long Context Generative AI (<https://arxiv.org/abs/2405.07719>)

<https://gitcode.com/cann>

Ulysses 与 Ring 的二维组合



USP 的思路是把设备组织成二维拓扑：**一个维度用 Ulysses 做 head 并行**，**另一个维度用 Ring Attention 做序列方向的 K/V 传递**。

从理论上讲，USP 统一了 Ulysses 和 Ring。但在实际部署中，是否采用 USP 要看**硬件的组网方式**。

并行方案汇总：计算结合通信的不同选择

方案	通信方式	优点	局限	适合场景
CFG 并行	all_gather	简单，收益直接	并行度固定为 2	使用 CFG 的扩散模型
Ulysses	All-to-All	工程简洁，单机常用	head / 尺寸有约束	单机多卡优先
Ring Attention	p2p send/recv	显存低，单轮 FA 短	p2p 复杂，依赖带宽	超长序列需评估
AllGather SP	AllGather	集合通信成熟，可 overlap	全局 K/V 显存高	高带宽 Fullmesh
UAA	动态 All-to-All	支持不规整输入	padding 和负载不均	业务输入尺寸多变
USP	All-to-All + p2p	扩展并行度	组网方式依赖强	跨节点或 head 不足

开发者如何选择并行方案

SP 的关键不是“切得越多越好”，而是**让序列切分方式、Attention 通信模式和硬件组网方式相互匹配。**

硬件平台	拓扑特征	优先策略	适用原因
A2	Fullmesh, 多卡互联较均衡	Ulysses SP	Ulysses 依赖 All-to-All, Fullmesh 更适合高频全互联通信
A3	Switch 组网, 通信路径受交换网络影响	Ulysses baseline, 必要时评估 Ring / Hybrid	Switch 组网下 All-to-All 可能受链路竞争和 rank 映射影响, Ring / Hybrid 可通过 K/V 交换和 overlap 缓解通信压力

判断项	结论
DiT / Attention 是否是主要瓶颈	是, 再优先评估 SP; 否则先优化 VAE、Text Encoder、I/O 或算子路径
拓扑是否适合 All-to-All	适合, 优先 Ulysses; 不适合, 评估 Ring / Hybrid
输入尺寸或 head 数是否满足整除约束	不满足, 启用 UAA
是否需要更高 SP degree	需要, 评估 Ring / Hybrid, 而不是只提高 Ulysses degree
最终判断标准	端到端时延、显存压力、通信占比、稳定性、组网方式匹配度

并行方案本质上是通信组织方式，没有脱离硬件组网方式的固定答案。



目录

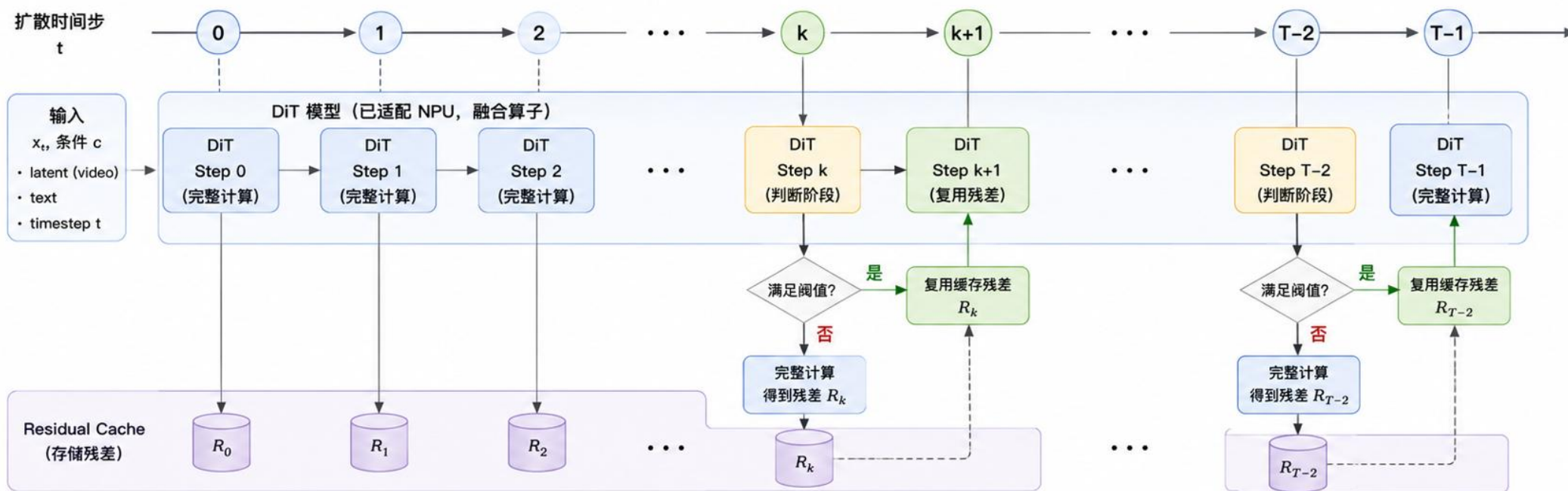
Part 1 Sequence Parallel

Part 2 Feature Cache

Feature Cache 的原理：为什么扩散模型可以做Cache

原理：扩散模型在采样阶段需要执行多轮去噪 step，每个 step 都要调用一次 DiT/Transformer 主干。以 HunyuanVideo 为例，一次step包括80次attention 和 MLP 计算，完整执行所有 step 的成本很高。

依据：相邻 timestep 的 latent、调制向量、block 输出通常变化平滑，存在大量“相似计算”。Feature Cache 的核心思想是用少量判断成本，换掉一部分完整 Transformer 前向计算。



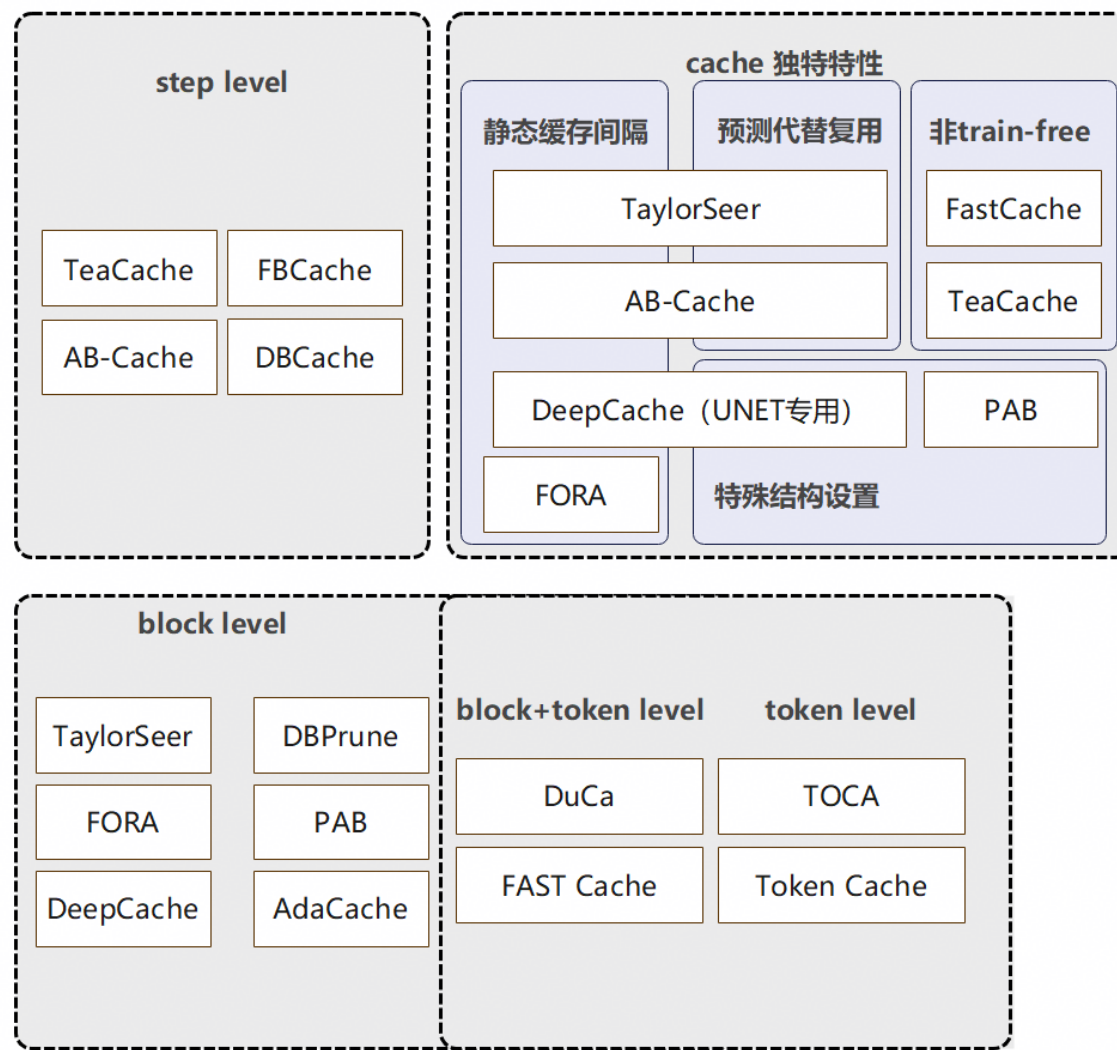
Feature Cache 的分类:

Feature Cache 可多维度体系化分类, 包括:

- 按缓存颗粒度;

Cache分类	原理	典型方案
Step-level	复用/预测整个扩散步骤 (step) 的计算结果	TeaCache/ FBCache
Block-level	复用/预测 Diffusion 中 block /Attention/mlp模块的输出结果,	TaylorSeer
token-level	复用 Transformer 中 单个 token 的计算结果	TOCA

- 按 step/block 计算原理;
- 按应用场景;
- 按训练依赖;
- 按缓存间隔设置。



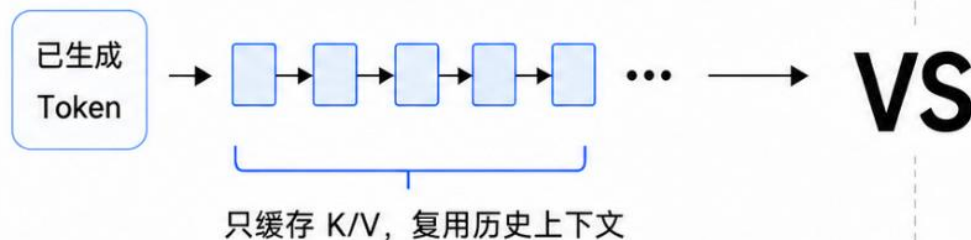
Feature Cache的优势与局限:

优势: Feature Cache 通过减少 **DiT** 完整前向次数实现推理加速，在长视频、高分辨率等重计算的生成场景中收益尤为突出，单卡场景可直接压缩主干执行耗时具备无需重新训练、仅需在推理侧轻量植入逻辑即可落地的工程优势，

局限: 和LLM的KVCache不一样，是一种有损加速方法。可通过**阈值、预热步数等参数**灵活平衡推理速度与生成质量

KV Cache (大语言模型自回归生成)

缓存的是 LLM 自回归生成中已生成 token 的 K/V，避免重复算历史上下文，从而加速后续生成。



Feature Cache (扩散模型推理)

缓存的是扩散模型相邻 denoising step 的残差或中间特征，用于跳过部分计算，从而减少完整推理步数。

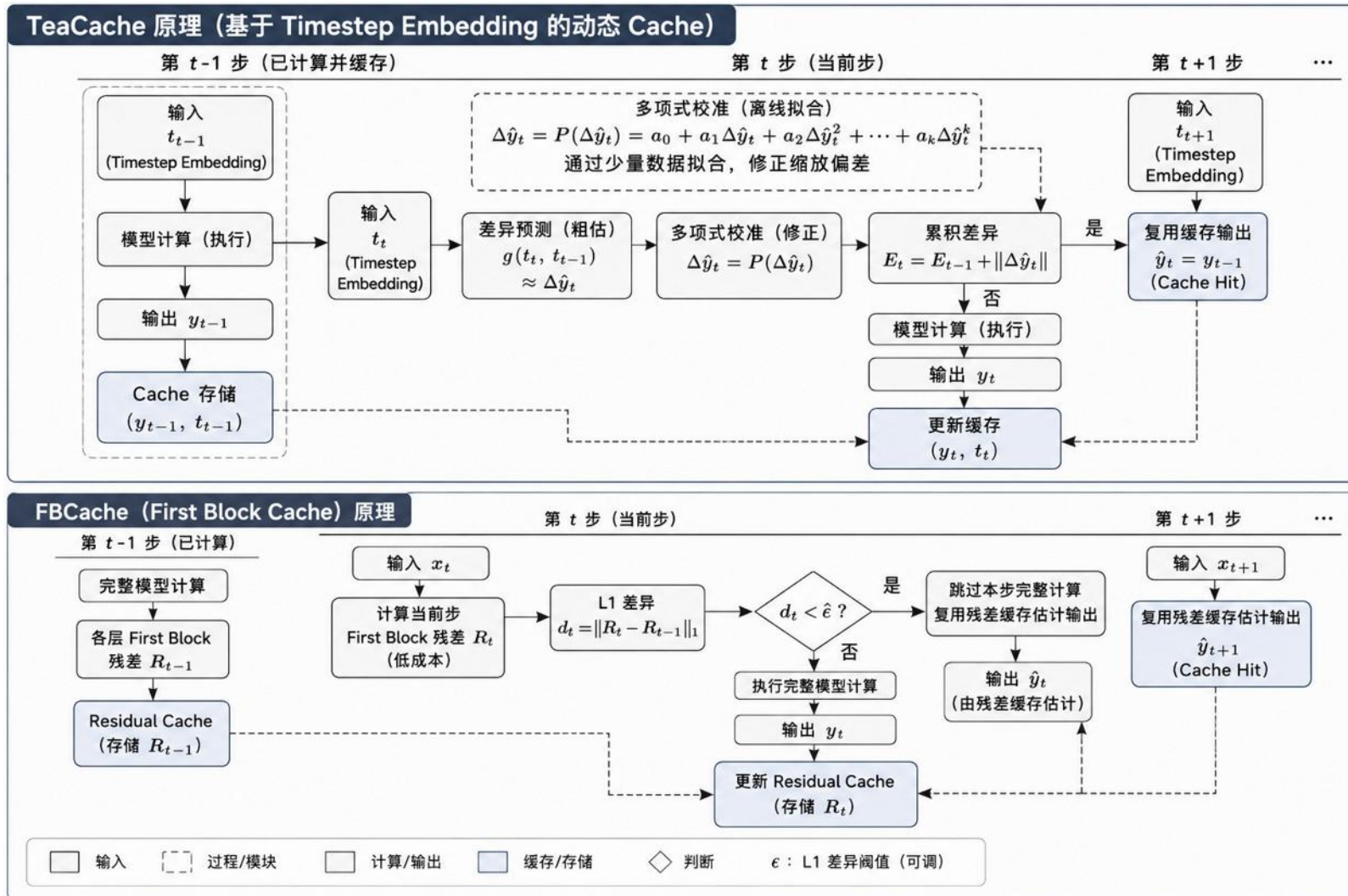


Feature Cache的典型方案：FBCache 和TeaCache

两者核心差异在判断逻辑：

TeaCache 采用时序调制输入做极低开销判断，预判并跳过非必要全量计算。

FBCache以 DiT 首个Block 输出为依据，通过相对 L1 差异阈值，决定是否跳过后续全量计算



recipes infer 中的Cache方案

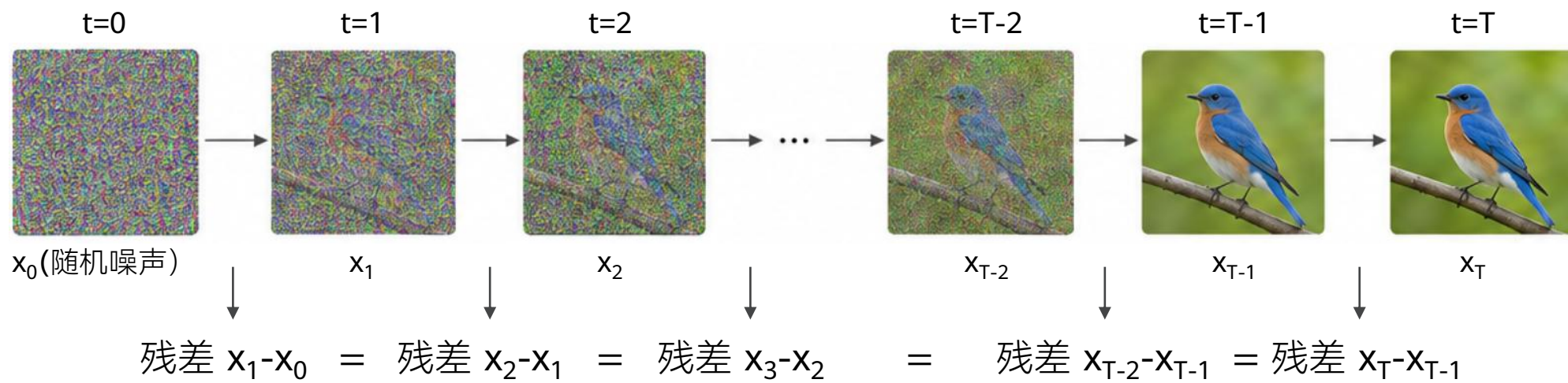
recipes infer 仓内置了两款轻量化 DiT 推理加速 Cache 方案 ——TeaCache 与 FBCache，为 hunyuanvideo/wan2.2模型提供单卡TeaCache /FBCache、8 卡 SP 与TeaCache /FBCache同时使能两类开箱即用配置文件。

```
dit_cache:
  method: "NoCache"                # [NoCache, FBCache, TeaCache, TaylorSeer]
  enable_separate_cfg: true
  # Uncomment the params block matching the method above; only that block is read.
  # params:
  #   # FBCache:
  #   rel_l1_thresh: 0.05
  #   #
  #   # TeaCache:
  #   rel_l1_thresh: 0.15
  #   coefficients: [733.226126, -401.131952, 67.5869174, -3.149879, 0.0961237896]
  #   warmup: 2
  #   #
  #   # TaylorSeer (needs 400GB+ host memory when offload=true at 720*1280*129):
  #   n_derivatives: 3
  #   skip_interval_steps: 4
  #   warmup: 3
  #   cutoff_steps: 1
  #   offload: true
```

rel_l1_thresh等核心调优参数，需要根据生成任务的质量要求灵活调整。

Feature Cache的典型方案: TaylorSeer

- FBCache和TeaCache直接复用历史DiT Timestep的残差，这里有一个核心假设：
DiT Timestep的输出是均匀的，即，相邻Timestep的输出是均等的



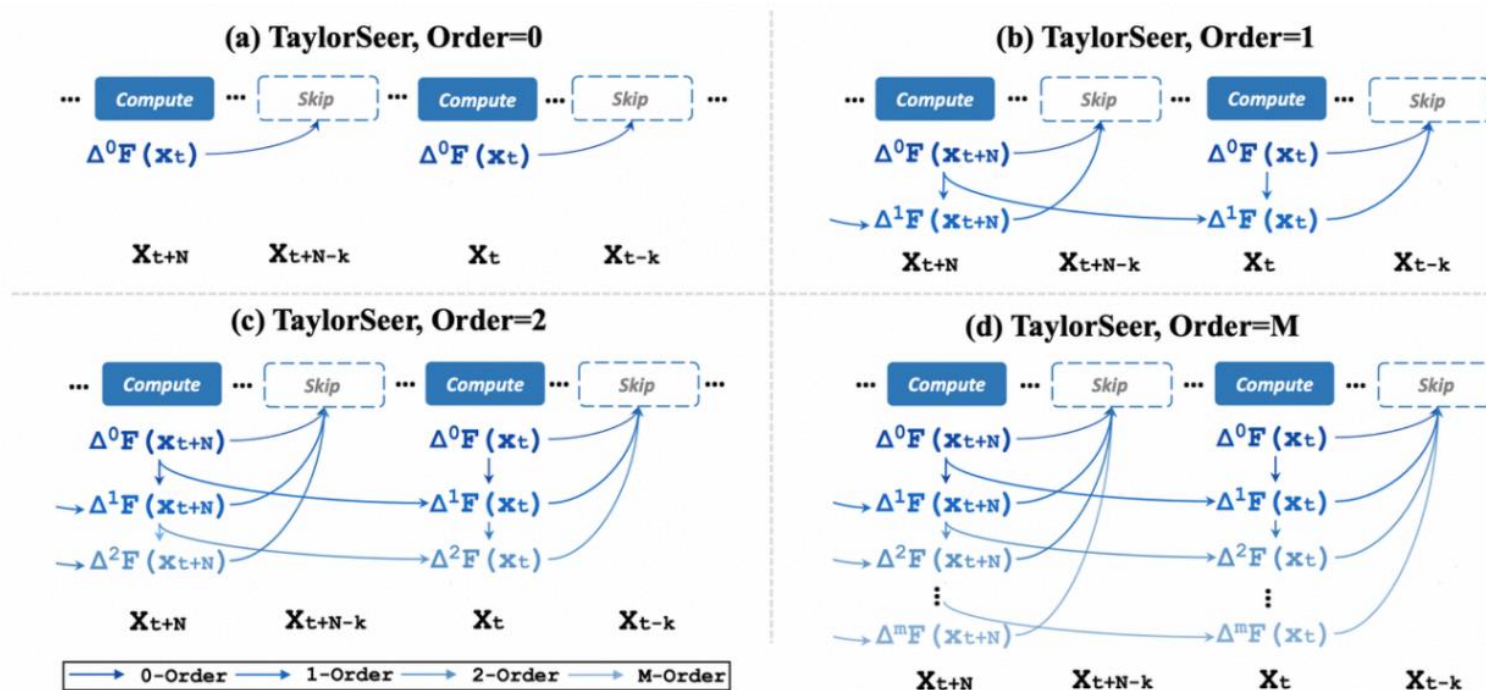
- 而在TaylorSeer眼里，DiT Timestep的输出是变化的，这个变化可以用一个数学工具来表示——**泰勒公式**

TaylorSeer——以预测代复用

- 回顾一下泰勒公式，在已知函数在某一点的各阶导数值的情况之下，泰勒公式可以用这些导数值做系数构建一个多项式来近似函数在这一点邻域中的值：

$$F(x) = f(x_0) + \frac{f'(x_0)}{1!}(x - x_0) + \cdots + \frac{f'(x_n)}{n!}(x - x_0)^n$$

- TaylorSeer的核心思想就是从“复用缓存”升级为“预测特征”
- TaylorSeer在缓存历史特征的同时，还会计算和缓存历史特征的导数



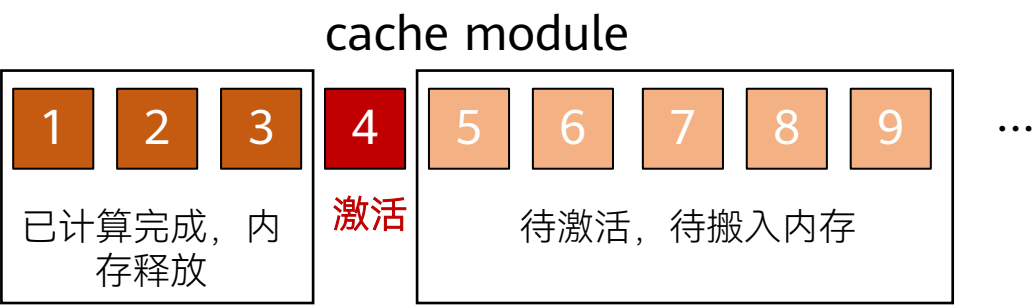
TaylorSeer——内存占用

TaylorSeer需要额外占用多大的内存？

序列长度	Head	Dim	推理精度	缓存阶数	模块数	内存占用
119K	24	128	BF16	3	80	163GB

TaylorSeer需要额外占用内存**163GB**，原模型在推理时的内存峰值约为60GB，在当前的NPU规格下这是难以接受的

TaylorSeer的计算有一个特点，虽然缓存了所有模块的导数，但**单次仅需激活单个模块**，因此可通过CPU Offload技术，将非活跃数据从 NPU 卸载至 CPU 内存或外存，从而降低 NPU 的内存占用。

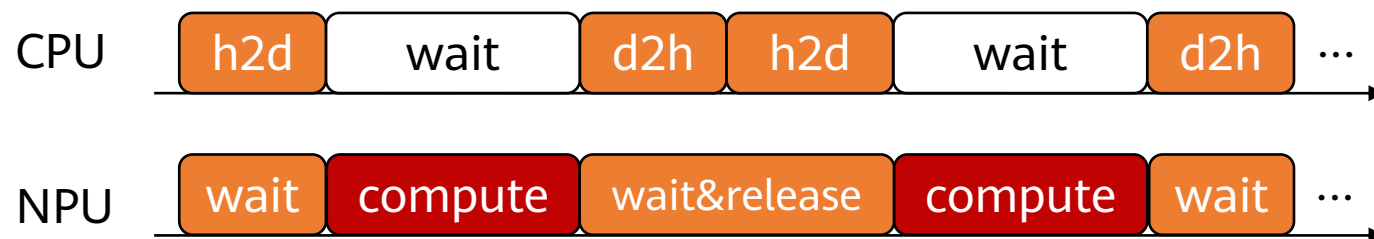


TaylorSeer——CPU offload

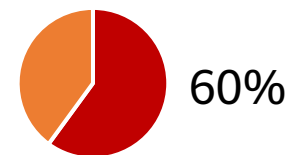
h2d(host to device): 将host(cpu)内存的内容搬运到device(npu)
d2h(device to host): 将device(npu)内存的内容搬运到host(cpu)

TaylorSeer本质上分为两个阶段:

1. **实际推理:** 包含了完整的DiT流程, 并且更新缓存的导数, 此部分涉及缓存内容读取 (h2d), 导数计算 (在NPU上进行), 储存计算结果 (d2h)



实际推理阶段

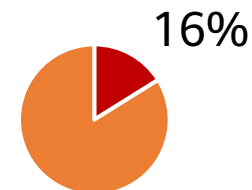


■ compute ■ free

2. **泰勒预测:** 基于缓存的导数, 计算各个模块的输出, 此部分涉及缓存内容读取 (h2d), 泰勒计算 (在NPU上进行), 释放NPU内存



泰勒预测阶段

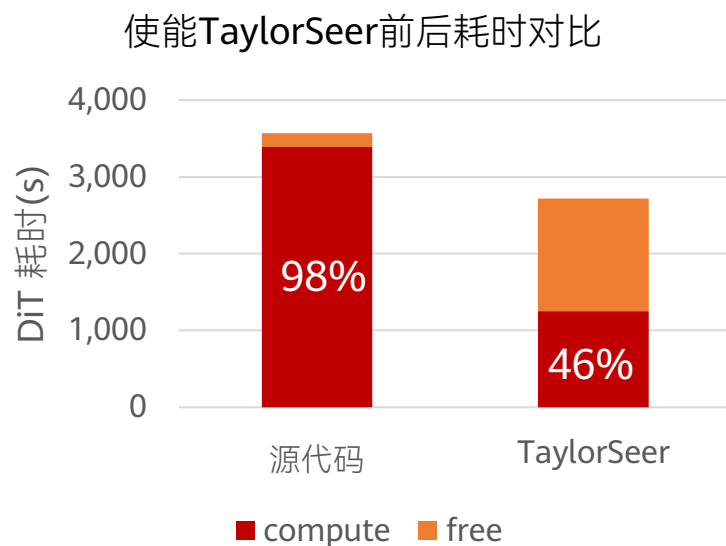


■ compute ■ free

TaylorSeer——搬算并行

现有问题:

TaylorSeer实现了推理加速, 但是NPU利用率大大降低, 产生大量free时间



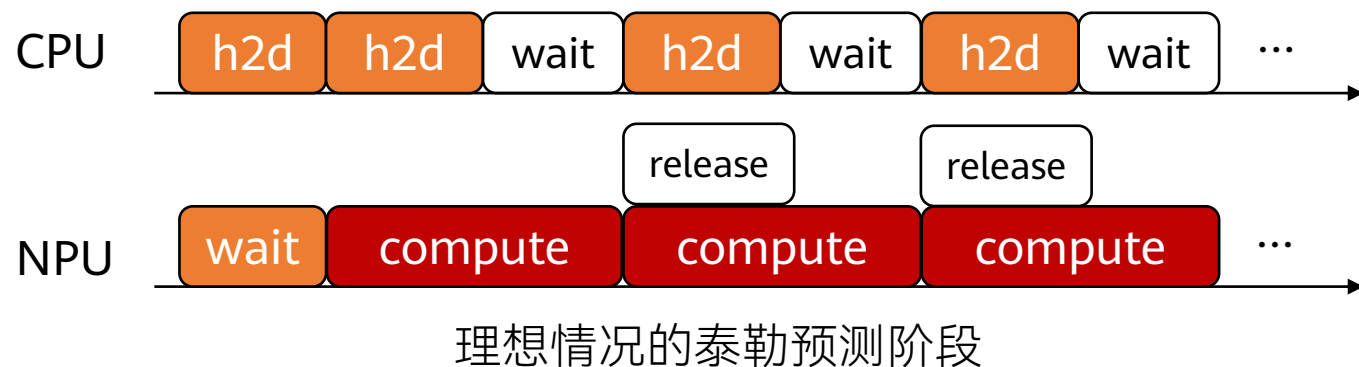
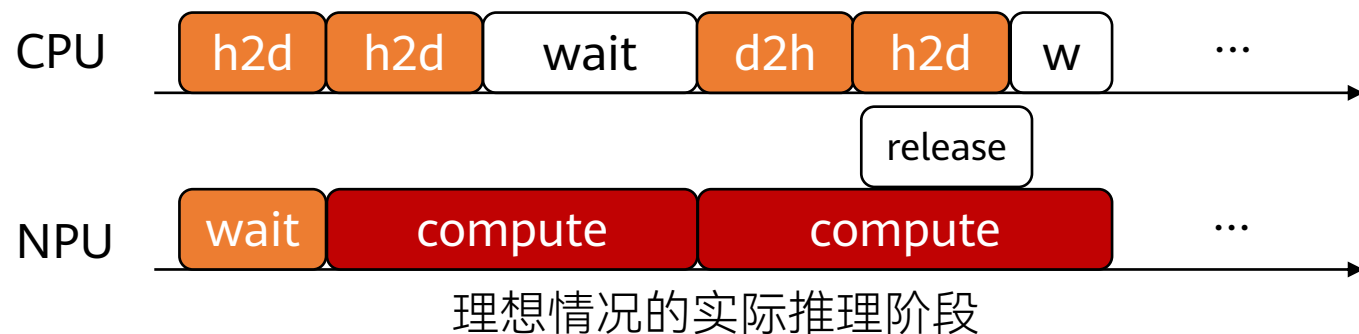
解决方案:

设计一套**搬算并行流程**, 让CPU搬运和NPU计算同步进行, 最大程度利用NPU资源

TaylorSeer——搬算并行

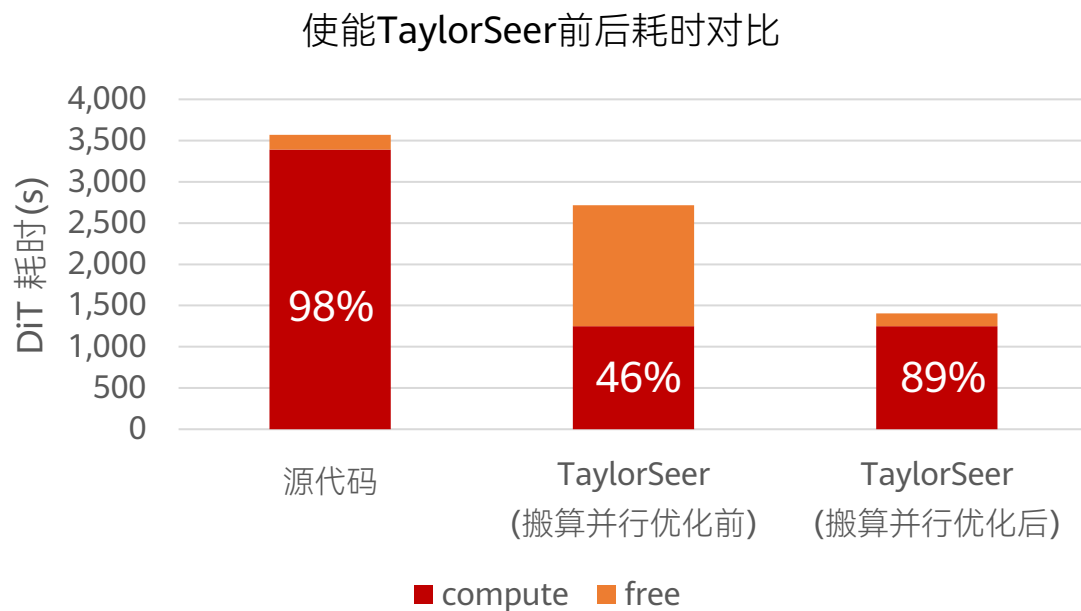
- **预取(prefetch)**: 提前将NPU计算所需的tensor从CPU搬运到NPU内存, 此搬运可以计算同时进行
- 使用**Stream**和**Event**来管理同步问题:
 1. 在CPU搬运完成前, 阻塞NPU计算, 避免NPU提前从内存中读写
 2. 在NPU计算完成前, 阻塞CPU搬运, 避免过度搬运, 使NPU OOM(Out Of Memory)

理想情况下, 搬算并行让搬运耗时完全被NPU计算掩盖, 如图:



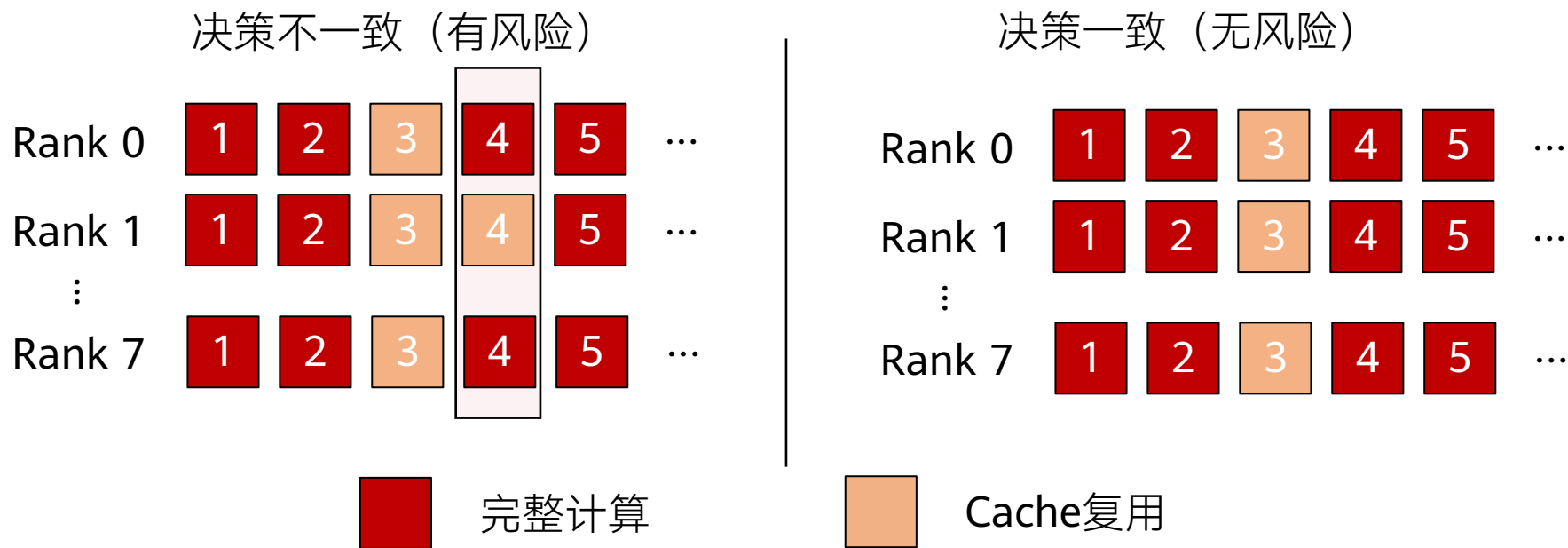
TaylorSeer——搬算并行

经过搬算并行优化后，NPU利用率获得了显著提升



Feature Cache+序列并行

Feature Cache在和序列并行相结合时，需要解决一个核心问题：**在每个时间步，各卡必须在“完整推理”和“缓存复用”之间决策一致**



- TaylorSeer使用固定策略，可以无缝适配序列并行；
- 而FBCache和TeaCache通过阈值判断，需要一次通信，收集各卡的损失取均值再与阈值比较，保证各卡的阈值判断结果一致。

Feature Cache可视化

Prompt: Close up of grapes on a rotating table. (旋转桌上葡萄的特写镜头)



baseline



FBCache(2.03x)



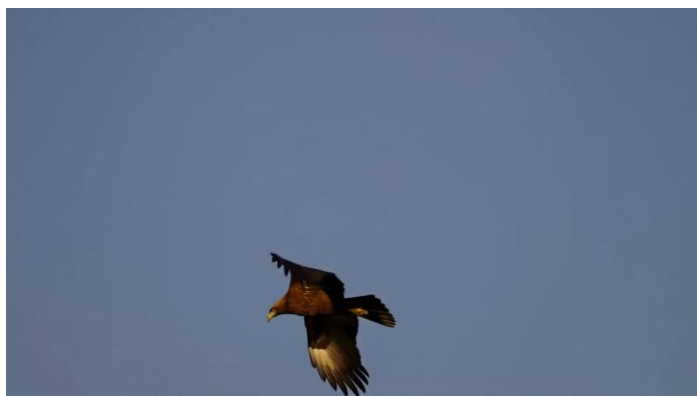
TeaCache(1.97x)



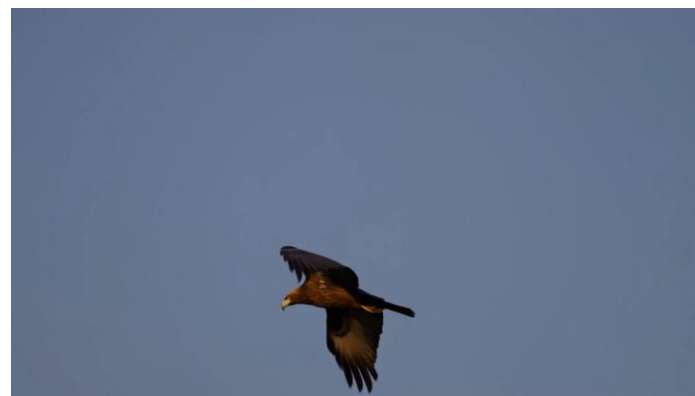
TaylorSeer(2.41x)

Feature Cache可视化

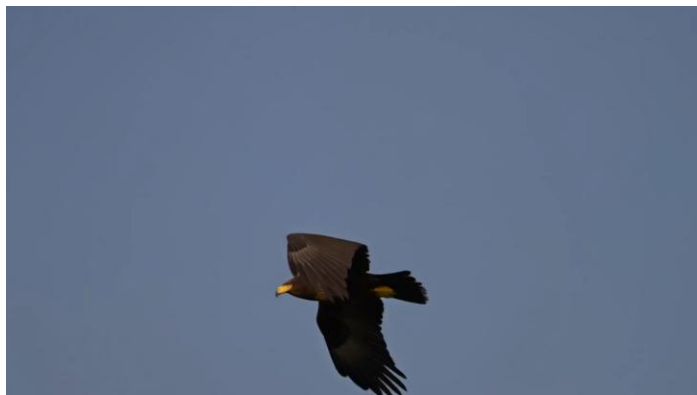
Prompt: a bird soaring gracefully in the sky. (一只鸟在天空中优雅地翱翔)



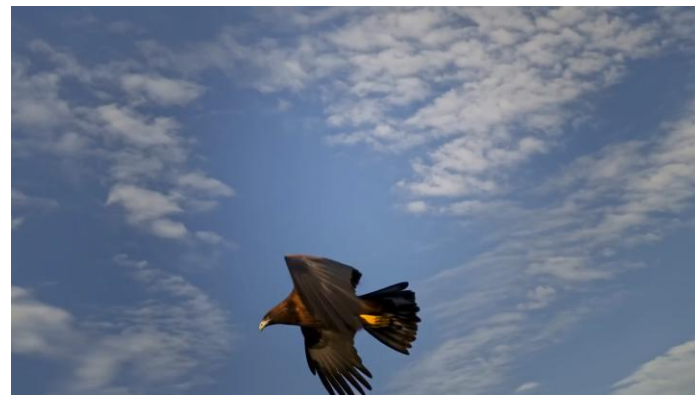
baseline



FBCache(2.03x)



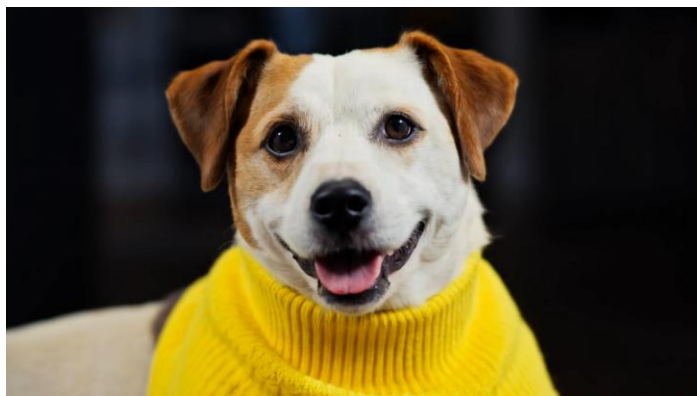
TeaCache(1.97x)



TaylorSeer(2.41x)

Feature Cache可视化

Prompt: happy dog wearing a yellow turtleneck, studio, portrait, facing camera, dark background.
(穿着黄色高领毛衣的快乐狗狗，摄影棚，肖像，面向镜头，深色背景)



baseline



FBCache(2.03x)



TeaCache(1.97x)



TaylorSeer(2.41x)

欢迎大家扫码加入我们的社区



cann recipes infer



微信交流群



飞书交流群

技术交流 & 在线答疑 & 行业动态 & 资源分享

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN